

VR-Apparatus: A Machine-Verified Apparatus for the Operational Methodology of the VR Cycle

Vitaly Reznik

May 2026

Contents

Part I — Position	2
I.1 What VR-Apparatus is	2
I.2 What VR-Apparatus is not	3
I.3 The recognition theme	3
I.4 Architectural summary	4
I.5 Acknowledgement	5
Part II — Architecture	5
II.1 Two apparatus modes	5
II.2 The IdentityNature distinction	6
II.3 Five-tier layered architecture	6
II.4 The two-register analogy	8
Part III — Mode A	8
III.1 The closure theorem	8
III.2 Productive triviality	9
III.3 Apparatus-structure-independence	10
III.4 Composition and identity	10
III.5 Mode A across the two apparatus modes	11
Part IV — Mode B	12
IV.1 The schema	12
IV.2 Factorisable: the canonical witness	12
IV.3 The spectrum of witnesses	13
IV.4 Self-witnessing	14
IV.5 Two-level Factorisable	14
IV.6 Mode B composition	15
IV.7 The Riesz extension as canonical Mode B instance	15
Part V — Apparatus instances and morphisms	16
V.1 Predicate-wrapping instances	16
V.2 Reference semantics instances	17
V.3 InterApparatusMorphism	17

V.4 VR-Forms integration	18
V.5 Numbers as hybrid	19
Part VI — Compositional algebra and findings	20
VI.1 Two parallel tracks	20
VI.2 Within-track composition	21
VI.3 Identity elements	21
VI.4 Axiom asymmetry between tracks	22
VI.5 The endomorphism constraint	23
VI.6 Productive triviality as systemic pattern	23
VI.7 Catalogued findings	24
Part VII — Position relative to existing frameworks	25
VII.1 Proof mining (Kohlenbach, Avigad)	25
VII.2 Constructive analysis (Bishop)	25
VII.3 Computable analysis (Weihrauch, Brattka)	26
VII.4 Reverse mathematics (Friedman, Simpson)	26
VII.5 Where VR-Apparatus fits	27
VII.6 What VR-Apparatus contributes	28
Acknowledgements	28
References	29
Appendix: Mapping from preprint Finding numbers to v1.0.0 Stage references	30

Part I — Position

I.1 What VR-Apparatus is

VR-Apparatus is the seventh work in the VR Cycle. It is a meta-work: it does not introduce new mathematical objects, new axioms, or new foundational positions. Instead, it formalises in Lean 4 the methodological apparatus that the prior six works of the cycle (VR, VR-Numbers, VR-Sets, VR-Forms, VR-Audit, VR-Sets-ZFA) used implicitly.

The apparatus is the discipline by which the VR Cycle distinguishes operational objects from classical ones, and by which classical results are connected to operational counterparts. Through the six prior works this discipline appeared in different concrete forms: as predicates over classical types (`IsComputableReal` on `Real`), as quotients by bisimulation (`OSetZFA = Quotient CoPSet.cobisim`), as syntactic translations (`translate_pi → isRealisable` in VR-Forms). These different forms were not collected into a single framework. They were applied, case by case, by the human and AI collaborators producing the cycle.

VR-Apparatus collects them. It introduces two apparatus typeclasses (`PredicateOperationality`, `ReferenceOperationality`), two transit modes (Mode A for operations that stay within the operational register, Mode B for operations that cross from classical to operational), and a five-tier layered architecture organising the methodology.

Sixty-eight public objects across twelve implementation files document the apparatus. Forty of these (fifty-nine percent) are axiom-free in the sense of Lean 4: their `#print`

axioms output is empty. The remainder sit at the standard mathlib ceiling [propext, Classical.choice, Quot.sound] or at intermediate tiers discovered during the work.

I.2 What VR-Apparatus is not

VR-Apparatus is not a new mathematical theory. It contains no new theorem about real analysis, set theory, arithmetic, or any other mathematical domain. The Riesz representation, the Hahn-Banach extension, the Anti-Foundation Axiom — all of these were proved in prior cycle works (VR-Audit, VR-Sets-ZFA). VR-Apparatus re-reads them through apparatus structures, but does not re-derive them.

VR-Apparatus is not a foundational position. It does not advocate for constructive mathematics over classical, or for set-theoretic foundations over type-theoretic. The cycle’s prior works were deliberately pluralistic: VR-Sets-ZFA proves AFA as a theorem in a coinductive setting, VR-Forms maintains ZF as the formal register, VR-Audit accepts mathlib’s classical machinery as a black box. The apparatus formalises the methodology of working across these positions, not any single position itself.

VR-Apparatus is not a programme of new audits. The Mode B schema, once formalised, suggests further applications: Banach-Steinhaus, the open mapping theorem, Stone-Weierstrass, the spectral theorem for compact self-adjoint operators. These remain future work. VR-Apparatus formalises the apparatus by which such audits would be structured, not the audits themselves.

VR-Apparatus is not a tutorial. It assumes familiarity with the VR Cycle’s prior works. Readers approaching the apparatus without that background will find the predecessor preprints helpful: VR. A Formal System (DOI 10.5281/zenodo.20324391) for the cycle’s ontological foundation, VR-Forms (DOI 10.5281/zenodo.20355939) for the two-register apparatus that motivated the abstraction, VR-Audit (DOI 10.5281/zenodo.20364111) for the first concrete Mode B instance.

I.3 The recognition theme

A theme runs through the apparatus work: recognition rather than invention. Several pieces planned as new abstractions turned out to be unnecessary once the apparatus was assembled.

The generic Register structure, proposed during scoping as a Tier 5 abstraction collecting apparatus instances by carrier type, was abandoned. VR-Forms already uses the term Register for its ontological-formal distinction, creating a name collision; and the “collection of instances by carrier type” concept is already provided implicitly by Lean’s typeclass system, which dispatches by type without explicit collection structures.

The DirectionalMorphism typeclass, proposed to handle the asymmetric formal-to-operational transit in VR-Forms, was also abandoned. The transit `translate_implies_realisable : ∀ t, translate_pi t → isRealisable t` matches the Mode B schema with trivial witness `W = fun _ => True` exactly. No new typeclass is required; the Mode B from VR-Audit already covers the VR-Forms transit. (This is recorded as Finding F11.)

The proposed cross-track composition theorems were not formalised: analysis traces through the discovery that no natural composition exists between the predicate track (predicate-wrapping + Mode B) and the reference track (reference semantics + InterApparatusMorphism). The two tracks are layered, not unified. (Finding F1.)

In each case the original plan called for new structure; the apparatus work, carried through to its conclusion, found that the structure was either already present (Mode B as a sufficiently general schema), or not needed (typeclass dispatch covering Register), or not present in the way envisioned (no cross-track composition). The apparatus is what remained after these recognitions.

I.4 Architectural summary

The apparatus, as it now stands, has five tiers:

1. **Apparatus instances** — PredicateOperationality $T\ P$ (the predicate-wrapping apparatus: an operational type T together with a predicate $P : T \rightarrow \text{Prop}$ distinguishing operational elements) and ReferenceOperationality $Q\ s$ (the reference semantics apparatus: a type Q of representatives with a setoid s , lifting to a quotient).
2. **Domain structures** — HasSeparabilityStructure T , providing the separability witnesses that enable the Mode B factorisability theorem for the analysis instances. Tier 2 is occupied so far only by the separability typeclass; future apparatus extensions may add parallel typeclasses for other domains.
3. **Morphism levels** — Mode A (operations preserving the operational predicate; closure under such operations), InterApparatusMorphism (representative-level maps between different setoids, lifting to maps between different quotients), Mode B (classical operations yielding operational results when an appropriate witness is supplied).
4. **Compositional algebra** — composition rules within each level (Mode A operations compose, IAM operations compose, Mode B operations compose with witness accumulation) and the recognition that no natural cross-level composition exists between the two tracks.
5. **Cross-instance recognition** — VR-Forms transit as a Mode B instance, numbers as hybrid subjects of both predicate-wrapping and reference semantics apparatuses depending on natural structure.

The remainder of the preprint develops these tiers in detail. Part II sets out the two apparatus modes and the identity nature distinction underlying the layered architecture. Part III treats Mode A. Part IV treats Mode B, including the canonical witness structure (Factorisable) discovered in Stage 4 of the v1.0.0 cycle. Part V documents the concrete instances and the cross-apparatus morphisms. Part VI develops the compositional algebra and the structural findings that emerged from it, including the headline observation of two parallel tracks. Part VII positions VR-Apparatus relative to existing methodological frameworks in proof mining, constructive analysis, computable analysis, and reverse mathematics.

The companion Lean 4 formalisation is published at DOI 10.5281/zenodo.20380344. Source repository: <https://github.com/inventor1975/VRCycle>, tag v1.7-vr-apparatus-1.0.0.

I.5 Acknowledgement

VR-Apparatus was developed using Claude Opus 4.7 (architectural review) and Claude Sonnet 4.6 (Lean implementation), in the Variant A collaboration pattern established by prior cycle works. The exploratory mode that produced the apparatus — discoveries through plan-then-code iteration, with multiple abstractions proposed and then recognised as unnecessary — depended on this collaboration. The recognition theme of Section I.3 reflects the work’s actual developmental shape, in which proposals were repeatedly revised against what the apparatus, when assembled, could and could not support.

Part II — Architecture

II.1 Two apparatus modes

The VR Cycle, across its six prior works, distinguished operational objects from classical ones in two structurally different ways. The distinction is not a matter of degree or complexity. It is categorical: each apparatus mode treats identity of objects through a different mechanism.

The first mode, **predicate-wrapping**, treats objects as points in a classical type, identified by their position in that type, with an operational predicate selecting the operational subcollection. Concretely, $\mathbb{R}$ is mathlib’s classical real numbers, identified by their position on the real line; `IsComputableReal x : Prop` selects those reals carrying an explicit computable approximation. Identity is given by the classical type. The predicate adds operational data without altering identity.

The second mode, **reference semantics**, treats objects as positions in a graph of references, identified by their bisimulation equivalence class. Concretely, `0SetZFA` is the quotient of `CoPSet` (coinductive pre-sets) by cobisimulation; two pre-sets are equal in `0SetZFA` exactly when they have the same members at every depth. Identity is not given externally; it is constituted by the structure of references themselves. The Quine atom $q = \{q\}$ exists not as an object with an attached property but as a particular position in the reference graph — a self-referencing node.

These two modes are formalised in VR-Apparatus as two typeclasses:

```
class PredicateOperationality (T : Type*) (P : T → Prop) : Prop
class ReferenceOperationality (Q : Type*) [s : Setoid Q] where
  membership : Quotient s → Quotient s → Prop
  ext : ∀ x y, (∀ z, membership z x ↔ membership z y) → x = y
```

The asymmetry of the definitions is intentional. `PredicateOperationality` is a marker class with no fields: it declares that the pair (T, P) is an apparatus, but operations on operational elements are type-dependent and cannot be uniformly characterised. The

closure of `IsComputableReal` under addition is a specific theorem about computable reals, not a general fact about predicate apparatuses; it must be proved case by case.

`ReferenceOperationality` has two fields: `membership` and `ext`. These are the operations available uniformly to every reference apparatus. Membership and extensionality are domain-independent. Whether the quotient classes are sets, graphs, or recursive process descriptions, the apparatus provides these operations.

II.2 The IdentityNature distinction

Underlying the two-mode distinction is a categorical fact about how identity is constituted. `VR-Apparatus` introduces an explicit indicator:

```
inductive IdentityNature : Type where
  | AsPoint      : IdentityNature
  | AsReference  : IdentityNature
```

`IdentityNature` is a usage indicator, not an intrinsic type property. The same Lean type can be used in different modes in different contexts. The natural numbers $\mathbb{N}$ appear in `VR-Sets-ZFA` as von Neumann ordinals (identity by membership graph position, `AsReference`) and in `VR-Audit` as counters and indices in computable sequences (identity by value, `AsPoint`). The hybrid nature of numbers — Finding F8 — reflects this: a given mathematical object may be the subject of multiple apparatus lenses, and apparatus choice depends on what natural structure the object carries in the context of use.

Each apparatus instance fixes an identity nature. For `PredicateOperationality T P`, identity is `AsPoint`: elements of `T` are identified by their position in the classical type. For `ReferenceOperationality Q s`, identity is `AsReference`: elements of the quotient `Quotient s` are identified by their equivalence class, which collapses references into structural positions.

```
def PredicateOperationality.identityNature ... : IdentityNature := .AsPoint
def ReferenceOperationality.identityNature ... : IdentityNature := .AsReference
```

This is not a typeclass with computational content: it is a tag recording the apparatus mode at use. The distinction matters for choosing which apparatus to apply to a given object. A real number, treated as a point on the real line, is naturally subject to predicate-wrapping. A set, treated as a structure of memberships, is naturally subject to reference semantics. The mismatch — attempting to treat a real number as a quotient class with set-theoretic membership — would require choosing an artificial relation. Stage 5 of the v1.0.0 cycle confirmed this empirically: the attempt to construct a `ReferenceOperationality` instance on Cauchy sequences (the representative type whose quotient is $\mathbb{R}$) failed not for technical reasons but because Cauchy sequences carry no natural membership relation. The apparatus was not constructed; the discipline of honest scope was preserved.

II.3 Five-tier layered architecture

The apparatus, when assembled across the six pieces of v1.0.0, exhibits a five-tier layered structure:

Tier 1 — Apparatus instances. The two typeclasses `PredicateOperationality` and `ReferenceOperationality`, together with their concrete instances. This tier provides the basic dispatch mechanism: a given object, together with its identity nature, selects an apparatus.

Tier 2 — Domain structures. Typeclasses that provide additional structure required by specific apparatus instances for specific purposes. `HasSeparabilityStructure T` is the first such typeclass, introduced in Stage 6 to make explicit the separability witness that the analysis Mode B instances (Hahn-Banach on Hilbert) implicitly require. Tier 2 is by nature domain-specific; future apparatus extensions may add parallel typeclasses for algebraic structures, process algebras, or other domains.

Tier 3 — Morphism levels. Three distinct levels of operation between apparatus instances:

- *Mode A* (intra-instance endomorphisms): operations preserving the operational predicate (predicate-wrapping case) or respecting the setoid equivalence (reference semantics case). Closure under such operations is established as a theorem in each case.
- *InterApparatusMorphism* (between reference instances): maps between different setoids that respect both equivalence relations. Lift to maps between the corresponding quotients via `Quotient.lift` and `Quotient.sound`.
- *Mode B* (predicate-track conditional transit): classical operations yielding operational results when accompanied by a sufficient witness. The witness in canonical form is `Factorisable PA PB f a` — existence of a computable function matching the classical operation on the specific operand. Part IV develops this in detail.

Tier 4 — Compositional algebra. Composition rules within each morphism level (Mode A compositions, IAM compositions, Mode B compositions with witness accumulation), and the structural finding that no natural cross-track composition exists. Identity elements at each level. The two-track separation — predicate track and reference track — emerges as a categorical observation, not an imposed design.

Tier 5 — Cross-instance recognition. The recognition that existing VR Cycle apparatus already covers certain transit patterns that initial scoping treated as requiring new structure. VR-Forms transit (`translate_implies_realisable`) is a Mode B instance with trivial witness; the planned generic `Register` and `DirectionalMorphism` abstractions were unnecessary. Numbers as hybrid subjects of multiple apparatus lenses fall under Tier 5: not a new framework but a recognition of how Lean’s typeclass system already accommodates the hybrid nature.

The tiers are layered, not hierarchical in a strict sense. Higher tiers depend on lower ones (Tier 3 morphisms operate on Tier 1 apparatus instances; Tier 4 compositional algebra is over Tier 3 morphisms). But not every apparatus instance requires every tier. Reference apparatuses (`OSet`, `OSetZFA`) involve Tiers 1, 3 (Mode A, IAM), and 4. Analysis apparatuses (`IsComputableReal` in `OperationalHilbertSpace`) involve Tiers 1, 2 (`HasSeparabilityStructure`), 3 (Mode A, Mode B), and 4. The architecture accommodates this variation; tiers are scaffolding for understanding, not mandatory components.

II.4 The two-register analogy

A reader familiar with VR-Forms will recognise an analogy. VR-Forms introduced a two-register structure: the formal register (syntactic descriptions without ontological commitment) and the operational register (concrete VR-Sets objects with explicit construction). The two registers are connected by the transit pattern `translate_pi` $\rightarrow$ `isRealisable`.

The two apparatus modes of VR-Apparatus correspond to these two registers, but at a different abstraction level. Predicate-wrapping formalises objects in the operational register: classical types with operational predicates selecting computable elements. Reference semantics formalises a different aspect of the operational register: the structural identity of sets as positions in reference graphs.

The two registers of VR-Forms and the two modes of VR-Apparatus are not in one-to-one correspondence. VR-Forms's formal register is itself a predicate apparatus ((`FormalTerm`, `isRealisable`) and (`FormalTerm`, `translate_pi`) — both `PredicateOperationality` instances, both `AsPoint` identity nature). The transit `translate_implies_realisable` is a Mode B instance from one predicate apparatus to another, not a transit between apparatus modes. This is the substance of Finding F11: the VR-Forms two-register apparatus reads through VR-Apparatus as two predicate instances connected by Mode B, with the underlying ontological register (`OSet`, `ZFSet`) being a separate reference apparatus.

Finding F12 records the resulting three-way contrast of identity natures within VR-Forms:

- (`FormalTerm`, `isRealisable`) — `AsPoint` (existential predicate apparatus, formal register)
- (`FormalTerm`, `translate_pi`) — `AsPoint` (specific predicate apparatus, formal register)
- `instRefOpPSet` (`PSet` with `PSet.setoid`) — `AsReference` (ontological register)

The transit `vr_forms_transit_isModeB0p` connects the two `AsPoint` instances. The ontological register, with `AsReference` identity, is the universe in which realisability claims land.

The architecture of VR-Apparatus, then, generalises the VR-Forms two-register insight: rather than two registers fixed by the cycle's ontology, the apparatus accommodates an open collection of apparatus instances, each carrying its own identity nature, connected by the morphism levels of Tier 3. The two registers of VR-Forms are recovered as a specific configuration of this more general architecture.

Part III — Mode A

III.1 The closure theorem

Mode A formalises the case in which apparatus transit is trivial: operations that stay within the operational register require no additional structural argument to be recognised as preserving operationality.

For the predicate-wrapping apparatus, this takes the form:

```
def PredicateOperationality.IsModeAOp
  {T : Type*} {P : T → Prop} [PredicateOperationality T P]
  (f : T → T) : Prop :=
  ∀ x : T, P x → P (f x)
```

An operation f is Mode A if it preserves the operational predicate P : every operational element maps to an operational element. The closure theorem states that such an operation lifts to a function on the operational subtype:

```
def modeA_liftFn ... (hf : IsModeAOp f) :
  {x : T // P x} → {x : T // P x} :=
  fun ⟨x, hx⟩ => ⟨f x, hf x hx⟩
```

The proof is a single application: hf provides the predicate preservation, the constructor `Subtype.mk` packages the result with its certificate. The lifting theorem `modeA_lift` : `(modeA_liftFn hf x).val = f x.val` reduces by `rfl`.

For the reference semantics apparatus, the analogous schema applies to congruences:

```
def ReferenceOperationality.IsModeAOp
  {Q : Type*} [s : Setoid Q] [ReferenceOperationality Q]
  (f : Q → Q) : Prop :=
  ∀ x y : Q, x ≈ y → f x ≈ f y
```

An operation on the representative type is Mode A if it respects the setoid relation. The lifting uses `Quotient.lift`:

```
def modeA_liftFn ... (hf : IsModeAOp f) :
  Quotient s → Quotient s :=
  Quotient.lift (Quotient.mk s ∘ f) (fun x y h => Quotient.sound (hf x y h))
```

The lifting theorem in the reference case reduces to `Quotient.lift` of a representative through `Quotient.sound`. The axiom profile is `[Quot.sound]` — the only axiom used is the quotient soundness rule.

III.2 Productive triviality

The closure theorems in both Mode A variants have proofs that compute to `rfl` or reduce to single applications of standard lemmas. This triviality is not incidental. It is the substance of the schema.

The mathematical content of Mode A does not lie in the proof of closure. It lies in *which operations satisfy the schema*. Showing that addition on $\mathbb{R}$ preserves `IsComputableReal` requires non-trivial mathematics: the sum of two computable reals is computable through an explicit construction (`VR.Audit.IsComputableReal_add`, established in `VR-Audit`). Showing that the singleton operation on `0SetZFA` preserves cobisimulation requires the structural argument that singleton respects equivalence (`0SetZFA.singleton_isModeA`, also trivial once singleton is defined to respect equivalence representatives).

The apparatus packages these specific facts:

```
theorem isComputableReal_add_isModeA :
  PredicateOperationality.IsModeAOp₂
    (P := VR.Audit.IsComputableReal) (· + ·) :=
  fun x y hx hy => VR.Audit.IsComputableReal_add hx hy
```

The Mode A wrapping reduces to applying the underlying mathematical theorem (`IsComputableReal_add`) to two predicate certificates. The apparatus does not add new content; it recognises that this particular operation belongs in the apparatus.

This pattern — trivial proof, substantive recognition — recurs through the apparatus. The operand-determines-operational theorem of Mode B, the lifting theorems of `InterApparatusMorphism`, the identity elements of Stage 3, the bridge theorem `separability_provides_factorisable` of Stage 6 — all reduce to one- or two-line proofs. Across the v1.0.0 cycle, eight such proofs accumulated. The apparatus structure makes extraction free: staying within the operational register costs nothing formally. The substance of an apparatus lies elsewhere — in the specific recognitions and in the structural relations between modes.

III.3 Apparatus-structure-independence

A finding from Stage 2 of v1.0.0 (Finding F10 in its detailed form): the Mode A schema does not require the apparatus typeclass in its signature. The body of `IsModeAOp` uses only the predicate `P` (or the setoid `s` in the reference case); the typeclass `[PredicateOperationality T P]` is not referenced in the body and is automatically excluded from Lean’s section-variable generalisation.

This is structurally significant. The apparatus framework (`PredicateOperationality`, `ReferenceOperationality` classes) adds *organisational* content: it tags pairs (T, P) or (Q, s) as apparatuses, records identity natures, provides namespace structure. The *mathematical* content of Mode A operates one level below: it concerns predicates and functions, not apparatus tags.

A consequence is that Mode A theorems can be stated and proved without invoking apparatus typeclasses. The apparatus serves to organise the methodology, not to gatekeep the mathematics. This property — apparatus-structure-independence — extends to Mode B (Stage 3 finding), to `InterApparatusMorphism` (Stage 2 finding), and indeed to the entire Tier 3 morphism layer. The apparatus is scaffolding around theorems that stand on their own.

III.4 Composition and identity

Mode A operations compose:

```
theorem IsModeAOp.compose
  (hf : IsModeAOp f) (hg : IsModeAOp g) :
  IsModeAOp (g ∘ f) :=
  fun x hx => hg (f x) (hf x hx)
```

The composition is again Mode A: predicate preservation is transitive. The proof is again a one-liner.

Mode A has an identity element. The identity function on T trivially preserves any predicate:

```
theorem IsModeAOp_id : IsModeAOp (id : T → T) := fun _ hx => hx
```

In the reference case, the identity element is the quotient construction `Quotient.mk s`, which trivially respects the setoid:

```
theorem IsModeAOp_quotientMk : IsModeAOp (Quotient.mk s) :=
  fun a b hab => Quotient.sound hab
```

The lift of this Mode A operation is the identity on the quotient (`modeA_liftFn_quotientMk_eq_id`), establishing that the identity element acts as expected.

These identity and composition laws make the collection of Mode A operations into a monoid (in the predicate case) or into a partial algebraic structure (in the reference case, where the lift's domain and codomain are both the quotient). They are not formalised as typeclass instances of `Monoid` or `Category` — that would be premature abstraction. They are formalised as concrete theorems giving identity and composition.

III.5 Mode A across the two apparatus modes

A subtle point, made explicit in Stage 2 of the v1.0.0 cycle, is that the Mode A schemas in the two apparatus modes operate at *different architectural levels*. The predicate-wrapping Mode A operates on operations $f : T \rightarrow T$ between elements of the classical type (representative level). The reference semantics Mode A operates on operations $f : Q \rightarrow \text{Quotient } s$ from representatives directly to quotient classes (mixed level).

This asymmetry is not a defect. It reflects the different identity constitutions: a predicate-wrapping object is a point with an attached certificate, so operations between points are the natural unit; a reference semantics object is an equivalence class, so operations that respect the equivalence and produce quotient classes directly are the natural unit. The asymmetry surfaces when one considers cross-apparatus maps (the topic of Part V): an operation from a representative type to a *different* quotient (different setoid) requires a third concept, `InterApparatusMorphism`. This is not subsumed by either Mode A.

Finding F2 (layered structure, not subset) records this. Mode A and `InterApparatusMorphism` are not in a containment relation: the former is a special case of the latter only at the level of representative-level conditions, not at the level of result types. A Mode A operation ($Q \rightarrow \text{Quotient } s$, same setoid) cannot be obtained in general by restricting an `InterApparatusMorphism`'s source-target setoids to be equal. The two operate at different points in the quotient construction. Part V develops this further.

Part IV — Mode B

IV.1 The schema

Mode B formalises the case in which apparatus transit is conditional: classical operations yielding operational results when an appropriate witness is supplied. The schema:

```
def IsModeB0p {A B : Type*} (PA : A → Prop) (PB : B → Prop)
    (W : A → Prop) (f : A → B) : Prop :=
    ∀ a : A, PA a → W a → PB (f a)
```

The components have specific roles. PA is the source operational predicate (which inputs are operational). PB is the target operational predicate (which outputs are recognised as operational). W is the witness condition: an additional predicate on inputs that, together with PA, ensures PB (f a). f is the classical operation itself.

The schema reads: for every operational input a (satisfying PA) that carries the witness W, the result f a is operational (satisfies PB).

The witness W is the structural heart of Mode B. It is what distinguishes Mode B from Mode A. In Mode A, the operation preserves the predicate by virtue of its definition: no extra information is required. In Mode B, the operation may *not* preserve the predicate in general — it is a classical operation, possibly relying on non-constructive machinery — but on inputs carrying the witness, the result remains operational.

The reading is what Stage 3 of v1.0.0 called the **operand-not-operation** principle: operability of the result is determined by the operand (by PA a and W a), not by the operation f itself. The operation may invoke any classical machinery; the witness on the operand routes operability through.

IV.2 Factorisable: the canonical witness

The natural question, once the Mode B schema is given, is what witnesses can be supplied. A witness W is any predicate on A such that $PA\ a \wedge W\ a \rightarrow PB\ (f\ a)$ holds. In principle, there are many such predicates. The simplest is $W = \text{fun } _ \Rightarrow \text{True}$: every operand trivially satisfies the witness, and the entire content of IsModeB0p collapses to $\forall a, PA\ a \rightarrow PB\ (f\ a)$. This is the form used in VR-Audit's `riesz_extension_isModeB0p`, where operability of the result is established directly without an explicit witness structure.

Stage 4 of v1.0.0 introduced the canonical witness: Factorisable.

```
def Factorisable {A B : Type*} (PA : A → Prop) (PB : B → Prop)
    (f : A → B) (a : A) : Prop :=
    ∃ g : A → B, (∀ x : A, PA x → PB (g x)) ∧ f a = g a
```

Factorisable PA PB f a asserts the existence of a *computable* function g that agrees with the classical operation f on the specific operand a and preserves the operational predicate on all operational inputs. This is what makes the witness *canonical*: it records, in the witness itself, the computational content that makes the transit succeed.

The connection to Mode B is direct:

```
theorem operand_determines_operational
  (a : A) (ha : PA a) (f : A → B)
  (hfact : Factorisable PA PB f a) : PB (f a) := by
  obtain ⟨g, hg_preserves, hfa_eq⟩ := hfact
  rw [hfa_eq]
  exact hg_preserves a ha
```

Three lines: extract the computable witness g , rewrite $f\ a = g\ a$, apply preservation. The operand-not-operation principle becomes a theorem. From this:

```
theorem IsModeB0p_of_factorisable :
  IsModeB0p PA PB (Factorisable PA PB f) f :=
  fun a ha hfact => operand_determines_operational f a ha hfact
```

Mode B with $W = \text{Factorisable PA PB } f$ is the canonical Mode B instance. The witness records the route by which operationality is preserved; the theorem reads this route off the witness.

IV.3 The spectrum of witnesses

The canonical witness `Factorisable` is not the only valid choice. Different Mode B instances of the VR Cycle use different witness structures:

Trivial witness ($W = \text{fun } _ \Rightarrow \text{True}$). Used in `riesz_extension_isModeB0p` and in `vr_forms_transit_isModeB0p`. When operationality of the result is established by other means — by the orthogonal projection identity in the Hilbert case, by the syntactic structure of `translate_pi` in the VR-Forms case — the witness need not carry the route explicitly. The trivial witness records that no additional content is required beyond PA.

Predicate witness ($W = \text{PA}$). Used implicitly when `IsModeA0p` is viewed as a degenerate `IsModeB0p`. If the operation preserves PA in itself ($\text{PA} \rightarrow \text{PA} \circ f$), the witness $W = \text{PA}$ is the same as PA, and the schema reduces to $\text{PA} \rightarrow \text{PA} \circ f$ — that is, Mode A. Stage 4 of v1.0.0 records this as `IsModeA0p_iff_IsModeB0p`.

Domain structure witness (W defined by a Tier 2 typeclass). The Hahn-Banach Mode B instance, taken in its richer Stage 6 form, uses `HasSeparabilityStructure E` as the witness route: separability of the underlying space provides the dense sequence through which factorisation runs. Stage 6’s `separability_provides_factorisable` theorem makes this explicit.

Canonical (factorisable) witness ($W = \text{Factorisable PA PB } f$). The most explicit form: the witness records the computable function through which the classical operation routes.

These witness forms are not in a strict order of generality. They serve different purposes. Stage 4’s `factorisable_implies_isModeB0p` records that *any* witness implying factorisability suffices for Mode B:

```
theorem factorisable_implies_isModeB0p {W : A → Prop}
  (hw : ∀ a, W a → Factorisable PA PB f a) :
```

```

IsModeB0p PA PB W f :=
fun a ha hwa => operand_determines_operational f a ha (hw a hwa)

```

This is the canonicity theorem in its general form: Factorisable is sufficient; any witness that implies factorisability is also sufficient. The four forms (trivial, predicate, domain structure, canonical) all factor through this.

A practical reading: when constructing a new Mode B instance, the choice of witness records the structural reason the transit succeeds. Trivial witness when the proof handles operationality directly. Domain structure witness when the typeclass machinery carries the route. Canonical witness when the route is itself the natural formulation.

IV.4 Self-witnessing

A point requiring care, recorded as Finding F6: when an operation f is globally Mode B with trivial witness ($W = \text{fun } _ \Rightarrow \text{True}$), the factorisability witness can be taken to be f itself. The factorisation reduces to $f \ a = f \ a$, which is rfl . The operationality preservation reduces to the global Mode B condition.

This is not circular. It is the recognition that, when an operation is *already known* to be operational on every operational input, the operation itself serves as its own factorisation witness. The mathematical content (why the operation is operational) lives inside the proof of the global Mode B condition. The factorisability structure (the existence of a computable g matching f on the operand) is satisfied by $g = f$ trivially.

The implication is methodological. When the mathematical work has already been done to establish global Mode B — as in `HahnBanachOperational_Hilbert` from `VR-Audit` — there is no benefit to constructing an artificial witness. The self-witnessing form is honest: the work is in the underlying theorem, not in the witness structure. The canonical form (Factorisable) is for cases where the witness records a genuinely separable computation, distinct from the operation itself.

In VR-Audit’s Hahn-Banach case, the canonical g would be the extended functional itself ($g(x) = \langle P_M(x), \xi \rangle$, the orthogonal projection composed with the Riesz inner product), the very function constructed in the proof. Witness and operation coincide. Stage 4 records this case as `riesz_extension_factorisable`, with the witness $g = \text{innerSL } \mathbb{R} \ \xi$ extracted from the proof — definitionally equal to the operation.

IV.5 Two-level Factorisable

Stage 6 of v1.0.0 recorded a structural observation: factorisability applies at *two levels* in the Mode B schema.

Level A (operand level): Factorisable $PA \ PB \ f \ a$ where the operand a is the input to the classical operation. The Hahn-Banach case: the operand is the functional $f : \text{OperationalNormableFunctional } E \ M$, the operation is the Riesz extension `riesz_extension_map`, the factorisable witness records that the extension is itself a classical operation routed through an operational function.

Level B (evaluation-point level): Factorisable applied at points of the *output* of the Mode B operation. The Hahn-Banach case again: the output is the extended functional $g = \text{riesz_extension_map } f$, which itself can be analysed as a function on E . At each dense sequence point $\text{denseSeq } n$, $g (\text{denseSeq } n)$ is factorisable through $f (P_M (\text{denseSeq } n))$ — the dense sequence point projects onto M , the projection lies in M , the projected point is applied under f , which is operational.

The two levels are not contradictory. They are different applications of the same Factorisable concept. Level A is the factorisability of the Mode B *operation as a whole*. Level B is the factorisability of the *result evaluated at separable points*. Stage 6’s bridge theorem `separability_provides_factorisable` connects them: separability of the underlying space (Tier 2 structure) provides the dense sequence at which Level B factorisability holds.

This two-level structure suggests that Mode B has more architectural richness than initially apparent. The Mode B schema treats operationality at the operand level; separability and similar domain structures provide operationality at the evaluation point level; both are mediated by Factorisable at different scales. Stage 6 left the full formalisation of Level B for VR-Audit (which re-derives `hchain` from `HahnBanach.lean` and would require cascading `mathlib` imports) to future work; the observation is recorded.

IV.6 Mode B composition

Mode B operations compose, but with a structural feature absent in Mode A: the witnesses accumulate. If f is Mode B with witness W_A and g is Mode B with witness W_B , then $g \circ f$ is Mode B with witness $\lambda a, W_A a \wedge W_B (f a)$:

```
theorem IsModeBOp.compose
  (hf : IsModeBOp PA PB W_A f)
  (hg : IsModeBOp PB PC W_B g) :
  IsModeBOp PA PC (fun a => W_A a ∧ W_B (f a)) (g ∘ f) := ...
```

The composed witness records the route through both operations: the operand satisfies W_A (sufficient for f), and its image under f satisfies W_B (sufficient for g). The accumulation reflects the chained structural conditions.

This contrasts with Mode A composition, where the witness conditions are trivial throughout. The contrast is the algebraic substance of Mode B: it admits composition but at the cost of richer witness structures. Sequential Mode B chains carry conjunctive witnesses recording the conditions at each step.

In practice, Mode B chains in the VR Cycle have remained short. The first VR-Audit (Hahn-Banach) is a single Mode B step. Longer chains — open mapping $\rightarrow$ spectral theorem, for instance, where multiple classical theorems are sequenced — would carry conjunctive witnesses requiring careful tracking. Stage 3’s `IsModeBOp.compose` formalises the algebraic structure; its application to long chains awaits further audits.

IV.7 The Riesz extension as canonical Mode B instance

The first VR-Audit, formalised in VR-Audit-1, establishes Hahn-Banach for operational Hilbert spaces. The Riesz representation theorem of `mathlib` is invoked as a black box;

the output is a vector ξ in the subspace M such that the functional f on M is given by $f(x) = \langle x, \xi \rangle$. The classical Hahn-Banach extension then takes the inner product with ξ over the full space E , producing $g(x) = \langle x, \xi \rangle$ for all $x \in E$.

This pattern reads through VR-Apparatus as a Mode B instance:

```
theorem riesz_extension_isModeB0p ... :
  IsModeB0p riesz_PA riesz_PB (fun _ => True) riesz_extension_map :=
  fun f _ _ => HahnBanachOperational_Hilbert f
```

The witness is trivial ($\text{fun } _ \Rightarrow \text{True}$); the operational content of the result is established directly by `HahnBanachOperational_Hilbert`, which exhibits the dense sequence values $g(\text{denseSeq } n) = f(P_M(\text{denseSeq } n))$.

The Stage 4 alternative form (`riesz_extension_isModeB0p'`) re-derives the same statement with explicit factorisable witness, demonstrating the spectrum of witnesses. Both forms coexist in the apparatus, with the same axiom profile and the same content; the difference is the witness structure.

The reading is what makes VR-Audit a paradigmatic Mode B application: the classical theorem is taken as black box, the operability of the result is shown by structural argument (separability of the space, explicit dense sequence values, operability of the underlying functional). Mode B captures this pattern, with Factorisable recording the route when explicit witness construction is desired.

For full mathematical detail of the Hahn-Banach extension via Riesz representation, the reader is referred to the VR-Audit preprint (DOI 10.5281/zenodo.20364111). The point of this Part is not to re-derive the analysis but to position the result within the apparatus.

Part V — Apparatus instances and morphisms

V.1 Predicate-wrapping instances

The predicate-wrapping apparatus has two concrete instances in the v0.1.0 cycle and additional related instances in v1.0.0:

IsComputableReal on $\mathbb{R}$. Established in VR-Audit:

```
def IsComputableReal (x : ℝ) : Prop :=
  ∃ (alg : ℕ → ℚ) (mod : ℕ → ℕ),
    ∀ n k, mod n ≤ k → |(alg k : ℝ) - x| ≤ 1/2^n
```

A real number is computable when it has an explicit Cauchy-style algorithm and modulus of convergence. The predicate selects the operationally meaningful subset of $\mathbb{R}$. The `PredicateOperationality ℝ IsComputableReal` instance is established in `Apparatus/Wrapping.lean` as a marker class instance.

OperationalHilbertSpace E typeclass. A type E carrying operational Hilbert space structure has: a dense sequence $\text{denseSeq} : \mathbb{N} \rightarrow E$, density denseSeq_dense , and computable inner products on the dense sequence. The typeclass packages this as apparatus content. Stage 6 of v1.0.0 decomposes `OperationalHilbertSpace` into `HasSepa-`

rabilityStructure (separability witness: denseSeq + density) and apparatus-specific content (inner_computable).

The two predicate apparatus instances differ in scope: `IsComputableReal` is a predicate on a fixed classical type ($\mathbb{R}$), while `OperationalHilbertSpace` is a typeclass on a universe-polymorphic type ($E : \text{Type}^*$). Both fall under the predicate-wrapping apparatus mode. Both have identity nature `AsPoint`.

V.2 Reference semantics instances

The reference semantics apparatus has two concrete instances in v0.1.0:

instRefOpPSet on PSet. The ZFC apparatus:

```
instance instRefOpPSet : @ReferenceOperationality PSet PSet.setoid where
  membership := ( $\cdot \in \cdot : \text{ZFSet} \rightarrow \text{ZFSet} \rightarrow \text{Prop}$ )
  ext := fun _ _ h => ZFSet.ext h
```

Here `PSet` is `mathlib`'s pre-set type, `PSet.setoid` is `mathlib`'s extensional equivalence on pre-sets, and `Quotient PSet.setoid = ZFSet`. The membership is `mathlib`'s `ZFSet.mem`. The apparatus records that $(\text{PSet}, \text{PSet.setoid})$ is a reference apparatus with ZFC's well-founded sets as its quotient. Axiom profile: `[propext, Quot.sound]` — the ZFC reference apparatus is *Classical.choice*-free.

instRefOpCoPSet on CoPSet. The ZFA apparatus:

```
instance instRefOpCoPSet : @ReferenceOperationality CoPSet CoPSet.instSetoid where
  membership := OSetZFA.Mem
  ext := fun x y h => OSetZFA.ext h
```

Here `CoPSet` is the coinductive parallel of `PSet` (built via `PFunctor.M`), `CoPSet.instSetoid` is cobisimulation, and `Quotient CoPSet.instSetoid = OSetZFA`. The membership is the apparatus membership of `OSetZFA`. Axiom profile: `[propext, Classical.choice, Quot.sound]` — the ZFA reference apparatus uses `Classical.choice` through the `PFunctor.M` infrastructure of `mathlib`.

A finding (F9): the axiom profiles of the two reference apparatuses differ, despite both occupying the same Tier 1 architectural slot. ZFC's `PSet` is inductively defined, with `mem_irrefl` provable; the apparatus is constructive at its level. ZFA's `CoPSet` is coinductive, built through the M-construction, which uses `Classical.choice` through `mathlib`'s encoding. The asymmetry is structural and irreducible: the apparatus framework does not choose between ZFC and ZFA; it accommodates both at their respective axiom costs.

V.3 InterApparatusMorphism

Stage 2 of v1.0.0 introduced the inter-apparatus morphism concept:

```
def InterApparatusMorphism {Q1 Q2 : Type*}
  [s1 : Setoid Q1] [s2 : Setoid Q2]
  (f : Q1 → Q2) : Prop :=
   $\forall x y : Q1, x \approx y \rightarrow f x \approx f y$ 
```

An `InterApparatusMorphism` is a function between representative types of two reference apparatuses (different setoids) that respects both equivalence relations. The lifting:

```
noncomputable def InterApparatusMorphism.lift {f : Q1 → Q2}
  (hf : InterApparatusMorphism f) :
  Quotient s1 → Quotient s2 :=
  Quotient.lift (fun q => [[f q]])
  (fun a b hab => Quotient.sound (hf a b hab))
```

Two canonical `InterApparatusMorphism` instances:

The von Neumann embedding of $\mathbb{N}$ into `PSet`. Stage 5 of v1.0.0:

```
theorem nat_vonNeumann_isInterApparatus :
  InterApparatusMorphism PSet.ofNat ...
```

The natural numbers embed into the ZFC universe through the standard von Neumann ordinal construction. The embedding respects the trivial equivalence on $\mathbb{N}$ (equality) and `PSet.setoid` on `PSet` (extensional equivalence). The IAM lift produces the standard ZFC embedding of $\mathbb{N}$ into `ZFSet`.

The ZFC-to-ZFA embedding. Stage 5 (re-reading of v0.1.0's `embedPSet_congr_modeA_pattern`):

```
theorem embedPSet_isInterApparatus :
  InterApparatusMorphism embedPSet
```

The embedding from pre-sets to coinductive pre-sets respects both extensional equivalence and cobisimulation. The IAM lift is `embed0Set`, the embedding of `0Set` (ZFC) into `0SetZFA` (ZFA). Stage 2's `embed0Set_eq_interApparatus_lift` records that the v0.1.0 `embed0Set`, originally constructed by direct `Quotient.lift`, is *definitionally* equal to the IAM lift of `embedPSet_isInterApparatus`. The apparatus reads existing structure; no rewriting is required.

The IAM lift uses only `Quot.sound` — neither `propext` nor `Classical.choice`. This is recorded as Finding F3: the IAM infrastructure occupies a sub-ceiling axiom tier [`Quot.sound`], between axiom-free objects and the standard [`propext`, `Quot.sound`] tier. Seven public objects of the v1.0.0 apparatus sit at this tier, all related to IAM lifting and to certain Mode A operations that factor through `Quotient.mk`. The structural reading: pure quotient algebra requires only the soundness axiom; propositional extensionality and classical choice enter only with concrete classical content (real number infrastructure, Riesz machinery, `PFunc.M` for coinductive types).

V.4 VR-Forms integration

Stage 1 of v1.0.0 records that VR-Forms's two-register transit is a Mode B instance with trivial witness:

```
instance instPredicateOpFormalTerm :
  PredicateOperationality FormalTerm isRealisable := {}

instance instPredicateOpTranslatePi :
  PredicateOperationality FormalTerm translate_pi := {}
```

```

theorem vr_forms_transit_isModeB0p :
  IsModeB0p translate_pi isRealisable (fun _ => True) id :=
  fun t h _ => translate_implies_realisable t h

```

The formal register of VR-Forms consists of two predicate apparatus instances on the same carrier `FormalTerm`: `translate_pi` (the direct VR-Sets operational predicate, naming concrete objects like `osetEmpty` and `omega_0Set`) and `isRealisable` (the existential realisability predicate, $\exists s : 0Set, P(s)$). The VR-Forms transit theorem `translate_implies_realisable` exactly fits the Mode B schema with the identity function as the operation and trivial witness: specific predicates imply existential predicates, and the apparatus already covers this case.

This is the recognition theme in its purest form: Mode B from the v0.1.0 cycle (introduced for the analysis case of Hahn-Banach) already covers the syntactic transit case of VR-Forms. No new abstraction is required. The “VR-Forms integration” that initial scoping proposed as a new Register concept is instead the recognition that VR-Forms is already a Mode B instance.

Finding F12 records the three-way identity nature contrast within the VR-Forms apparatus structure:

- `instPredicateOpFormalTerm`: `AsPoint` (formal register, existential predicate apparatus)
- `instPredicateOpTranslatePi`: `AsPoint` (formal register, specific predicate apparatus)
- `instRefOpPSet`: `AsReference` (ontological register, ZFC reference apparatus)

The Mode B transit connects the two `AsPoint` instances. The `AsReference` instance is the universe in which the realisability claims (the existential apparatus) land.

V.5 Numbers as hybrid

Stage 5 of v1.0.0 records that mathematical number systems admit multiple apparatus lenses, with applicability depending on the natural structure each type carries. The reconnaissance:

```

-- ℝ as predicate apparatus: natural
instance : PredicateOperationality ℝ IsComputableReal := {} -- from v0.1.0

-- ℝ as reference apparatus: artificial
-- Cauchy sequences have no natural membership relation.
-- A ReferenceOperationality instance with synthetic membership
-- would be methodologically dishonest. Not constructed.

```

The carrier for a reference apparatus on the reals would be `CauSeq ℚ abs` (Cauchy sequences modulo equivalence), with `Quotient CauSeq.equiv = Real.Cauchy.Cauchy abs`. `Mathlib` provides `Real.equivCauchy : ℝ ≈ Real.Cauchy.Cauchy abs`, so the carrier exists. But Cauchy sequences are not sets; they carry no natural membership relation. A `ReferenceOperationality` instance would require choosing an artificial

membership (such as `fun _ _ => False` or some contrived relation), producing a structure with no mathematical content.

Stage 5 recorded the bridge:

```
theorem cauchy_abs_isQuotient ... := rfl
def real_cauchy_bridge ... := Real.equivCauchy
```

And the corresponding natural number case:

```
theorem nat_vonNeumann_isInterApparatus :
  InterApparatusMorphism PSet.ofNat ...
```

The contrast is the finding (F8). $\mathbb{N}$ admits a reference apparatus naturally: von Neumann ordinals carry membership, and the embedding into `PSet` is an `InterApparatusMorphism`. $\mathbb{R}$ admits a predicate apparatus naturally (`IsComputableReal`), but no reference apparatus.

This is honest scope: the apparatus framework accommodates multiple lenses on the same type when natural structure permits, but does not force lenses where none exist naturally. Lens applicability depends on what structure the type already carries, not on the apparatus framework’s preferences.

Lean’s typeclass system already supports the hybrid case: multiple instances on the same type can coexist ($\mathbb{N}$ carries both `Add` and `Mul` simultaneously; in apparatus terms it could carry both a predicate apparatus and a reference apparatus). The apparatus framework does not need additional structural extension to accommodate this. The hybrid nature of numbers is a documented observation; no new typeclass is introduced for it.

Part VI — Compositional algebra and findings

VI.1 Two parallel tracks

The headline structural finding of the apparatus, recorded as F1, is that the apparatus has two parallel tracks rather than a unified single architecture.

Predicate track: `PredicateOperationality` apparatus instances, connected by Mode A (predicate-preserving endomorphisms) and Mode B (classical operations with witnesses). The instances live on classical types with operational predicates; the operations are between elements of those types (representative level, since predicate-wrapping does not quotient).

Reference track: `ReferenceOperationality` apparatus instances, connected by Mode A (setoid-respecting endomorphisms on representatives lifting to quotients) and `InterApparatusMorphism` (maps between different setoids, lifting to maps between different quotients). The instances live on representative types with setoids; the operations involve both representative and quotient levels.

The two tracks do not naturally compose with each other. There is no canonical morphism from a predicate apparatus to a reference apparatus, nor in the reverse direction. The reasons are structural:

Predicate apparatus $\rightarrow$ reference apparatus: a predicate-wrapping operational element ($t : T, ht : P\ t$) has identity `AsPoint` (position in T). To produce an element of a reference apparatus (quotient class with `AsReference` identity), one would need to construct a representative and provide a congruence proof. There is no canonical way to do this from the predicate; the construction depends on what the predicate means and what the reference apparatus is. The mapping is a specific theorem, not a general apparatus construction.

Reference apparatus $\rightarrow$ predicate apparatus: a reference operational element is a quotient class $\llbracket q \rrbracket$. To produce a predicate-wrapping element, one needs to assign a classical type representation and verify the operational predicate. Again specific, not general.

This non-composability is recorded in Stage 3 of v1.0.0 as the §5 non-composability observation, with three concrete cases documented: Mode B $\circ$ IAM (predicates and setoids live in different type universes), Mode B $\circ$ Reference Mode A (separate tracks, no bridge), Predicate Mode A $\circ$ IAM (different type universes again).

The two tracks are not unified into a single algebra. The apparatus accommodates both, with each having its own composition rules and identity elements. This is the honest architectural picture: a single framework with two parallel tracks, not a unified theory.

VI.2 Within-track composition

Each track has its own composition rules.

Predicate track: - `IsModeAOp.compose`: Mode A operations compose to Mode A operations. - `IsModeBOp.compose`: Mode B operations compose to Mode B operations with conjunctive witnesses. - Mode A $\circ$ Mode B and Mode B $\circ$ Mode A: Mode B with appropriate witness reorganisation (Stage 3 details).

Reference track: - Reference Mode A composition: setoid-respecting operations compose. - `InterApparatusMorphism.compose`: IAM operations compose (Stage 2). - `InterApparatusMorphism.lift_compose`: the lift of a composition equals the composition of lifts.

Cross-track within each apparatus mode: - Predicate Mode A and Mode B compose via the implication `IsModeAOp_iff_IsModeBOp` (Mode A with witness PA is Mode B with the same witness; the schemas are interconvertible). - Reference Mode A and IAM compose via `IsModeAOp_of_interApparatus` (when source and target setoids coincide, the IAM reduces to Reference Mode A).

The composition algebra within each track is rich enough to support sequential apparatus reasoning. The composition algebra between tracks does not exist, and the apparatus does not pretend otherwise.

VI.3 Identity elements

Identity elements at each level were formalised in Stage 3:

```

-- Predicate Mode A identity
theorem PredicateOperationality.IsModeAOp_id : IsModeAOp id := fun _ hx => hx

-- Reference Mode A identity (Quotient.mk acts as identity at the lift level)
theorem ReferenceOperationality.IsModeAOp_quotientMk :
  IsModeAOp (Quotient.mk s) := fun a b hab => Quotient.sound hab

-- Reference Mode A identity certificate
theorem modeA_liftFn_quotientMk_eq_id :
  modeA_liftFn IsModeAOp_quotientMk = id := ...

-- InterApparatusMorphism identity
theorem InterApparatusMorphism.id_isInterApparatus :
  @InterApparatusMorphism Q Q s s id := fun _ _ h => h

-- Mode B identity (trivial witness)
theorem IsModeBOp_id : IsModeBOp PA PA (fun _ => True) id := fun _ ha _ => ha

```

Each identity is a one-liner. The substance is again recognition: the apparatus structure makes identity elements present at every level, all reducing to definitional identities.

VI.4 Axiom asymmetry between tracks

Finding F9, recorded across Stages 5 and the comprehensive Stage Polish: the predicate track and the reference track exhibit systematic axiom asymmetry.

The predicate track, when its types involve real number infrastructure ($\mathbb{R}$, Cauchy abs, IsAbsoluteValue), inherits the standard ceiling [propext, Classical.choice, Quot.sound] even for trivially-proved objects. The reason is *type elaboration*: type-class synthesis pulls classical machinery through the field structure of $\mathbb{Q}$ and the order structure of $\mathbb{R}$, regardless of whether the proof itself uses classical machinery. A rfl proof of an equation about real numbers still inherits the ceiling because $\mathbb{R}$'s definitional infrastructure depends on it.

The reference track, when its types involve set-theoretic infrastructure (PSet, $\mathbb{N}$, Quotient), can be axiom-free or sit at the sub-ceiling tier [Quot.sound]. The ZFC apparatus (instRefOpPSet) uses [propext, Quot.sound]. The ZFA apparatus (instRefOpCoPSet) uses the full ceiling, through the PFunctor.M coinductive construction. The IAM infrastructure uses [Quot.sound] alone. The natural number von Neumann embedding is axiom-free.

The pattern: analysis-based apparatus inherits classical machinery through type elaboration; set-theoretic apparatus can be free of it or use only Quot.sound. The asymmetry is not a defect of the apparatus; it mirrors the deeper VR Cycle pattern in which the ontological foundation (VR-Sets, ZFC apparatus) can be constructive or near-constructive, while the analysis (VR-Audit) accepts mathlib's classical machinery as the appropriate tool for its task. The apparatus framework preserves this distinction in its axiom profiles.

The total axiom profile of v1.0.0 (68 public objects): - 40 objects (59%) at [] (axiom-free) - 7 objects (10%) at [Quot.sound] - 4 objects (6%) at [propext, Quot.sound] - 17 objects (25%) at [propext, Classical.choice, Quot.sound]

The four-tier profile is finer-grained than the three-tier profile of v0.1.0; the new [Quot.sound] tier emerged from IAM infrastructure in Stage 2. Finding F3 records this as a structural observation: pure quotient algebra is intermediate between axiom-free reasoning and the standard ceiling.

VI.5 The endomorphism constraint

A type-level finding from Stage 3, recorded as F10: the `ReferenceOperationality.IsModeA0p` schema is *strictly endomorphic*. The schema’s signature requires the codomain to be `Quotient s` where `s` is the source setoid; a cross-apparatus map with different source and target setoids cannot be expressed as Reference Mode A.

This is enforced by Lean’s type system. Attempting to write

```
IsModeA0p (g ∘ f) : IsModeA0p Q1 s1 (g ∘ f) -- where g : Q2 → Quotient s2
```

results in a type mismatch: `g ∘ f` produces values in `Quotient s2`, not `Quotient s1`, but `IsModeA0p Q1 s1` requires the codomain to be `Quotient s1`. The cross-apparatus composition must be expressed through `InterApparatusMorphism`, not Reference Mode A.

The architectural reading: Reference Mode A and `InterApparatusMorphism` are not in a containment relation. They occupy different positions in the morphism level (Tier 3). Reference Mode A is strictly endomorphic (intra-instance). `InterApparatusMorphism` is the cross-instance morphism. The two are connected by `IsModeA0p_of_interApparatus` (when source and target coincide), but neither subsumes the other.

This was recognised in Stage 2’s revised understanding: the original PLAN.md framing “Mode A ≤ `InterApparatusMorphism` ≤ Mode B partial order” was inaccurate; the levels are layered, not subset-ordered. F2 records this: layered structure, not subset hierarchy.

VI.6 Productive triviality as systemic pattern

Across the v1.0.0 cycle, eight or more theorems were established with proofs reducing to single applications or `rfl`:

1. `modeA_lift` (v0.1.0): `(modeA_liftFn hf x).val = f x.val` by `rfl`.
2. `operand_determines_operational` (Stage 4): three lines.
3. `IsModeB0p_of_factorisable` (Stage 4): one-liner.
4. `factorisable_implies_isModeB0p` (Stage 4): one-liner.
5. `InterApparatusMorphism.lift_mk` (Stage 2): `rfl`.
6. `IsModeA0p_of_interApparatus` (Stage 2): one-liner.
7. `separability_provides_factorisable` (Stage 6): one-liner.
8. Identity elements (Stage 3): all one-liners.
9. `IsModeB0p_id`, `id_isInterApparatus`, `IsModeA0p_id`: one-liners.

The pattern is not coincidence. Each “trivial” theorem corresponds to a structural identity: the apparatus structure has been defined so that the closure or recognition becomes a one-line consequence.

The substance is the underlying definitions, not the proofs. The apparatus is calibrated so that the *mathematical work* (proving that a specific operation is operational, that a specific space is separable, that a specific witness factorises) lives in the prior theorems (`IsComputableReal_add`, `HahnBanachOperational_Hilbert`, the `mathlib` definitions of `DenseRange` and `Quotient.sound`). The apparatus *recognises* these as fitting its schemas; the recognition is trivial; the underlying work is substantive.

This pattern, recorded across the v1.0.0 cycle as cumulative “productive triviality” observations, is part of what makes apparatus reasoning natural in Lean. The typeclass system and quotient infrastructure are designed for such reductions; the apparatus framework exploits them without resistance.

VI.7 Catalogued findings

Twelve findings emerged across the v1.0.0 cycle. They are catalogued here as a single list for reference:

F1 — Two parallel tracks (predicate, reference). No natural cross-track composition. Apparatus is not a unified theory.

F2 — Layered structure, not subset hierarchy. Mode A and IAM operate at different architectural levels (quotient vs representative); neither subsumes the other.

F3 — Sub-ceiling axiom tier [`Quot.sound`] discovered. Pure quotient algebra is intermediate between axiom-free reasoning and the standard ceiling. Seven public objects sit at this tier.

F4 — Operand-not-operation theorem. Operationality of result determined by operand, not by operation. Formalised as `operand_determines_operational`.

F5 — Spectrum of witnesses. Mode B admits witnesses from trivial (`fun _ => True`) through canonical (`Factorisable`); apparatus does not enforce a single witness form.

F6 — Self-witnessing. When global Mode B holds, the operation factorises through itself; witness and operation coincide.

F7 — Two-level Factorisable. Factorisability applies at operand level (the Mode B operation as a whole) and at evaluation-point level (results at separable points); both mediated by `Factorisable` at different scales.

F8 — Lens applicability depends on natural structure. Numbers admit multiple apparatus lenses, but not all lenses apply naturally to all number systems. $\mathbb{N}$ admits both predicate and reference lenses (von Neumann ordinals); $\mathbb{R}$ admits predicate but not reference (Cauchy sequences carry no natural membership).

F9 — Axiom asymmetry between tracks. Predicate track inherits classical ceiling through type elaboration involving analysis infrastructure; reference track (set-theoretic) can be axiom-free or sub-ceiling. Structural property.

F10 — Endomorphism constraint type-enforced. Reference Mode A schema requires source and target setoids to coincide; cross-apparatus maps must use `InterApparatusMorphism`. Enforced by Lean’s type system, not by external convention.

F11 — Generic Register abstraction unnecessary. VR-Forms transit fits Mode B schema directly. The proposed Tier 5 abstraction was dropped; the recognition is that v0.1.0’s Mode B already covers VR-Forms transit.

F12 — Three-way identity nature contrast in VR-Forms. Two `AsPoint` instances (formal register: `isRealisable`, `translate_pi`) and one `AsReference` instance (ontological register: `instRef0pPSet`). The Mode B transit connects the two `AsPoint` instances.

These twelve findings represent the structural content of the v1.0.0 cycle beyond the formalisation itself. They are the *recognitions* that the apparatus work produced.

Part VII — Position relative to existing frameworks

VII.1 Proof mining (Kohlenbach, Avigad)

Proof mining, developed by Ulrich Kohlenbach and collaborators [Kohlenbach 2008; Gerhardy & Kohlenbach 2008], extracts computational content from classical mathematical proofs. The technique applies proof interpretations — Gödel’s *Dialectica* interpretation in particular [Avigad & Feferman 1998] — to convert classical proofs into constructive proofs with explicit witnesses or moduli of convergence. Functional analysis, including the Hahn-Banach theorem and ergodic theorems, has been treated extensively.

VR-Apparatus differs from proof mining in scope and method. VR-Apparatus does not transform proofs. It classifies operations by whether their inputs carry operational witnesses; the proofs themselves remain as `mathlib` provides them, treated as black boxes. The Hahn-Banach theorem in VR-Audit invokes `mathlib`’s classical proof through `HahnBanachOperational_Hilbert`; VR-Apparatus reads this through Mode B, with the operational content captured in the operand witness, not extracted from the proof.

A more accurate positioning: VR-Apparatus is *upstream* of proof mining. Mode B characterises *when* an operational transit succeeds — i.e., when the operand carries sufficient witness for the classical operation to yield an operational result. Proof mining addresses *how* the witness can be extracted from a proof. The two programmes are compatible but operate at different levels.

A reader interested in extracting explicit moduli for Mode B operations would turn to proof mining. A reader interested in recognising which operations admit operational versions, and what structural witnesses are required, would turn to VR-Apparatus.

VII.2 Constructive analysis (Bishop)

Bishop’s constructive analysis [Bishop 1967; Bishop & Bridges 1985] rejects classical choice throughout. Theorems are proved with explicit constructions; existence proofs

require explicit witnesses. The mathematical content is in many cases identical to classical analysis, but the proofs differ structurally.

VR-Apparatus does not reject classical choice. The standard mathlib ceiling [propext, Classical.choice, Quot.sound] is accepted, with recognition of how often it can be avoided. Forty of sixty-eight public objects are axiom-free; seven sit at the sub-ceiling tier [Quot.sound]; the remaining twenty-one inherit the standard ceiling, primarily through analysis infrastructure.

The relation to Bishop is that VR-Apparatus formalises a tracking discipline: predicates select operational subsets, witnesses route operationality through classical operations, and the apparatus distinguishes operations by which witnesses they require. Bishop’s programme avoids classical machinery; VR-Apparatus accepts it while tracking what depends on what. The discipline is predicate-tracking, not choice-avoidance.

A reader committed to fully constructive mathematics would find VR-Apparatus less restrictive than required. A reader willing to accept classical mathematics while tracking computational content where it exists would find VR-Apparatus’s approach compatible with the broader mathematical culture.

VII.3 Computable analysis (Weihrauch, Brattka)

Computable analysis, in the Type-Two Effectivity framework [Weihrauch 2000; Brattka, Hertling & Weihrauch 2008], assigns computability content to operations on real numbers, function spaces, and similar objects. The framework includes a hierarchy of computability classes (Weihrauch reducibility) and detailed analysis of when classical theorems admit computable versions.

VR-Apparatus does not assign computability classes to operations. It asks whether operations *preserve* an operational predicate (Mode A) or admit operational results given sufficient witnesses (Mode B). The question is different: not “what is the computability cost of this operation?” but “does this operation belong in the operational register?”

The frameworks are compatible. A function that is computable in Weihrauch’s sense (Type-2 computable) would presumably satisfy appropriate Mode A or Mode B schemas if formalised in apparatus terms. But the apparatus does not formalise Type-2 computability directly. It uses Lean’s typeclass system to express apparatus instances; the underlying notion of “computable” is whatever the predicate `IsComputableReal` (or its analogues) makes explicit.

A reader interested in detailed complexity stratification of operations would turn to computable analysis. A reader interested in coarse classification (operational/classical, Mode A/Mode B) and in formal verification of the classification would turn to VR-Apparatus.

VII.4 Reverse mathematics (Friedman, Simpson)

Reverse mathematics, surveyed comprehensively by Simpson [Simpson 2009], asks which set-existence axioms are required to prove mathematical theorems. The Big

Five subsystems (RCA_0 , WKL_0 , ACA_0 , ATR_0 , $\Pi^1_1\text{-CA}_0$) form a hierarchy, with many classical theorems located precisely.

VR-Apparatus does not ask about set-existence axioms. It asks about operational tracking. The two programmes are orthogonal: reverse mathematics studies which axioms suffice for *existence*; VR-Apparatus studies which operations preserve *operationality*. A theorem provable in RCA_0 may or may not yield to apparatus reading; the question is independent.

A connection exists in the axiom profile of v1.0.0. The four-tier profile (axiom-free, [Quot.sound], [propext, Quot.sound], standard ceiling) is reminiscent of the reverse-mathematics stratification, in that it locates apparatus components by which foundational primitives they depend on. But the primitives are different: reverse mathematics uses second-order arithmetic axioms; VR-Apparatus uses Lean kernel axioms. The stratifications are not in correspondence.

A reader interested in axiomatic strength of theorems would turn to reverse mathematics. A reader interested in apparatus profiles of formalisations in dependent type theory would turn to VR-Apparatus.

VII.5 Where VR-Apparatus fits

The frameworks surveyed differ in their questions: proof mining asks how to extract content; Bishop asks how to do without choice; computable analysis asks at what computational cost; reverse mathematics asks at what axiomatic cost. VR-Apparatus asks: by what discipline does a working mathematician track which of their objects are operational?

The question is methodological rather than foundational. Working mathematicians — particularly those engaged with formalisation — do in fact track operational status informally. A real number used in a numerical computation is treated differently from a real number used in an existence proof. A set defined by an explicit construction is treated differently from a set whose existence is established via choice. The tracking is usually informal, embedded in mathematical practice rather than in formal frameworks.

VR-Apparatus formalises this tracking discipline. The apparatus typeclasses (`PredicateOperationality`, `ReferenceOperationality`) make explicit the apparatus instances at play. The morphism levels (`Mode A`, `Mode B`, `InterApparatusMorphism`) make explicit the transit patterns. The compositional algebra makes explicit how apparatus operations chain. The findings catalogue structural observations about the resulting framework.

The Lean typeclass system is the natural formalism for this work. Instance search corresponds to apparatus lookup. Typeclass hierarchy corresponds to apparatus architecture. The decomposition of `OperationalHilbertSpace` into `HasSeparabilityStructure` plus apparatus-specific content (Stage 6 finding) is the kind of refactoring that typeclass systems support natively. The apparatus framework is implementable as typeclasses, not as a separate metalanguage.

This is the methodological position of VR-Apparatus: it formalises the operational-

tracking discipline of working mathematics, implemented through Lean’s typeclass system, validated on the six prior works of the VR Cycle, with twelve structural findings catalogued from the v1.0.0 development. The position is modest: not a foundation, not a programme, not a new theory. A tool for recognising what is already implicitly being done, made explicit and machine-verifiable.

VII.6 What VR-Apparatus contributes

Three contributions, of differing scope:

A working formalism for operational-tracking discipline. The apparatus is implementable in Lean 4 using only typeclasses, predicates, and quotient operations. No new logical machinery is introduced. The 68 public objects of v1.0.0 demonstrate the framework on concrete cases drawn from the VR Cycle.

Twelve structural findings. The cycle development produced twelve findings, ranging from foundational (F1 — two parallel tracks) through specific (F8 — lens applicability depends on natural structure) to engineering (F3 — sub-ceiling axiom tier). The findings catalogue the structural shape of apparatus discipline.

A reference cycle of audits. The v1.0.0 cycle, together with v0.1.0, provides a worked example of apparatus discipline across multiple domains (analysis, set theory, syntactic translation, numbers). Future audits — Banach-Steinhaus, open mapping, spectral theorem — would proceed within this framework, with apparatus structures dictating what witnesses to construct and how to read the resulting Mode B schemas.

The contribution is to make explicit, in machine-verifiable form, the apparatus that the VR Cycle was using implicitly. The apparatus itself is not new; it has been used for centuries in working mathematics. What is new is its formalisation as a layered framework in Lean 4 with explicit twelve structural findings.

Acknowledgements

VR-Apparatus was developed between 24 and 25 May 2026, as a continuation of the VR Cycle work begun on 15 May 2026. The cycle as a whole — seven works across foundations (VR, VR-Numbers, VR-Sets), apparatus (VR-Forms), applied audit (VR-Audit), non-well-founded extension (VR-Sets-ZFA), and the present meta-work (VR-Apparatus) — was carried out by a single author over an eleven-day period.

The development used AI assistance throughout, in the Variant A collaboration pattern established by prior cycle works: Claude Opus 4.7 served as the architectural reviewer (planning stages, accepting or revising plans, reviewing stage reports, identifying findings) and Claude Sonnet 4.6 served as the Lean implementer (coding individual stages, running builds, producing axiom audits). The author retained architectural and methodological authority: every plan was approved before coding, every report was reviewed, every finding was discussed. The role of AI assistance was as a multiplier of the author’s architectural vision — accelerating implementation and surfacing structural observations that might otherwise have remained implicit — not as a substitute for that vision.

The Variant A pattern was applied consistently across the v0.1.0 and v1.0.0 cycles of VR-Apparatus, comprising approximately fifteen stage plan-and-report cycles between Opus and Sonnet, with the author overseeing each stage. The twelve findings catalogued in the preprint emerged through this collaborative process: some predicted in scoping, others surfacing only during implementation (notably Finding F3, the sub-ceiling [Quot.sound] tier, and Finding S1-A, the recognition that generic Register abstraction was unnecessary).

The temporal compression of the cycle — eleven days for seven works with machine-verified Lean formalisations and companion preprints — is itself worth noting. Comparable programmes in constructive or computable analysis have historically required years of community effort. This is not a comparison of intellectual depth or accumulated content; mature research programmes have decades of accumulated work that no single eleven-day effort can match. It is a comparison of *efficiency profile*: the Variant A workflow with present-generation AI assistance supports a rate of formal development that was not previously available to a single researcher.

References

- Avigad, J. & Feferman, S. (1998). Gödel’s functional (‘Dialectica’) interpretation. In *Handbook of Proof Theory* (S. Buss, ed.). Elsevier, pp. 337–405.
- Bishop, E. (1967). *Foundations of Constructive Analysis*. McGraw-Hill.
- Bishop, E. & Bridges, D. (1985). *Constructive Analysis*. Springer.
- Brattka, V., Hertling, P. & Weihrauch, K. (2008). A tutorial on computable analysis. In *New Computational Paradigms* (S.B. Cooper et al., eds.). Springer, pp. 425–491.
- Gerhardy, P. & Kohlenbach, U. (2008). General logical metatheorems for functional analysis. *Transactions of the AMS*, 360(5):2615–2660.
- Kohlenbach, U. (2008). *Applied Proof Theory: Proof Interpretations and their Use in Mathematics*. Springer.
- Moura, L. de & Ullrich, S. (2021). The Lean 4 theorem prover and programming language. In *Automated Deduction — CADE 28* (A. Platzer & G. Sutcliffe, eds.). Lecture Notes in Artificial Intelligence 12699. Springer, pp. 625–635.
- Reznik, V. (2026). VR. A formal system. Zenodo. DOI: 10.5281/zenodo.20324391.
- Reznik, V. (2026). VR-Numbers. Zenodo. DOI: 10.5281/zenodo.20352239.
- Reznik, V. (2026). VR-Sets. Zenodo. DOI: 10.5281/zenodo.20354628.
- Reznik, V. (2026). VR-Forms. Zenodo. DOI: 10.5281/zenodo.20355939.
- Reznik, V. (2026). VR-Audit. Zenodo. DOI: 10.5281/zenodo.20364111.
- Reznik, V. (2026). VR-Sets-ZFA. Zenodo. DOI: 10.5281/zenodo.20369346.
- Reznik, V. (2026). VR-Apparatus: Lean 4 formalisation. Zenodo. DOI: 10.5281/zenodo.20380344.

Simpson, S.G. (2009). *Subsystems of Second Order Arithmetic*, 2nd ed. Cambridge University Press.

The mathlib4 Community (2024). Mathlib4: A unified library of mathematics formalised in Lean 4. <https://github.com/leanprover-community/mathlib4>.

Weihrauch, K. (2000). *Computable Analysis: An Introduction*. Springer.

Appendix: Mapping from preprint Finding numbers to v1.0.0 Stage references

For researchers reading both this preprint and the v1.0.0 Lean cycle source code, the following table maps the preprint’s F-numbered findings to the internal S-prefixed stage references used in the source files and stage reports.

Preprint	Stage reference	Content
F1	S3-A	Two parallel tracks
F2	S2-A	Layered structure (quotient vs representative)
F3	S2-B	Sub-ceiling [Quot.sound] tier
F4	S4-A	Operand-not- operation theorem
F5	S4-A extension	Spectrum of witnesses
F6	S4-B	Self-witnessing
F7	S6-B (S6 architectural observation)	Two-level Factorisable
F8	S5-A	Lens applicability depends on natural structure
F9	S5-B (Stage 5 unexpected finding)	Axiom asymmetry between tracks
F10	S3-B	Endomorphism constraint type-enforced
F11	S1-A	Generic Register abstraction unnecessary
F12	S1-B	Three-way identity nature contrast

Source code reference: github.com/inventor1975/VRCycle, tag v1.7-vr-apparatus-1.0.0, files VRCycle/Apparatus/*.lean and the comprehensive Apparatus.lean module documentation header.